BSc. SOFTWARE ENGINEERING

COMPONENT BASED SOFTWARE ENGINEERING

SSE 2302

Component-Based Software Engineering (CBSE)

INTRODUCTION

Learning Outcomes

Upon successful completion of this course, the student should be able to:

1. Describe the role of Component-Based Software Engineering (CBSE) within the software life cycle.
2. Apply key elements and common methods for CBSE.
3. Design component-based software systems using well-structured design methods.
4. Describe, Compare, contrast and evaluate structured, Object Oriented, data Oriented and formal approaches to component modeling.
5. Conduct a review of CBSE requirements and using best practices to determine the quality of a CBS.
6. Demonstrate the capacity to use a range of software tools in support of CBS.
7. Analyze a software component problem and be able to design and implement an effective program structures to solve it, including appropriate modularity, separation of abstraction and implementation concerns, use of standard design patterns to solve recurring design problems, and use of standard libraries and COTS.
8. Discuss component composition and integration

INTRODUCTION AND DEFINITION

- **Component-Based Software Engineering (CBSE)** is a process that focuses on the design and development of computer-based systems with the use of reusable software components.
- It not only identifies candidate components but also qualifies each component's interface, adapts components to remove architectural mismatches, assembles components into a selected architectural style, and updates components as requirements for the system change.
- The process model for component-based software engineering occurs concurrently with *component-based development*.

Cont.:

- Component-based software engineering (CBSE) is an approach to software development emerged in the 1990's that relies on the reuse of entities called 'software components’.
- It emerged from the failure of object-oriented development to support effective reuse.
- Single object classes are too detailed and specific. Components are more abstract than object classes and can be considered to be stand-alone service providers.
- They can exist as stand-alone entities.

Example

- Suppose a university wants to develop a **Student Management System**. Instead of developing everything from the beginning, developers could reuse components for:
- User authentication
- Student registration
- Payment processing
- Report generation
- Email/SMS notifications
- Database access
- Document generation
- These components are integrated to form the complete system.

CBSE essentials

- **Independent components** specified by their interfaces.
- **Component standards** to facilitate component integration.
- **Middleware** that provides support for component inter-operability.
- **A development process** that is geared to reuse.

software engineering design principles:

Apart from the benefits of reuse, CBSE is based on sound **software engineering design principles**:

- Components are independent so do not interfere with each other;
- Component implementations are hidden;
- Communication is through well-defined interfaces;
- One components can be replaced by another if its interface is maintained;
- Component infrastructures offer a range of standard services.

Characteristics of Software Components

- A good software component normally possesses several important characteristics.

1 Well-defined functionality: A component should have a clear purpose. For example, a **Payment Component** should focus primarily on payment-related services rather than performing unrelated tasks.

2 Encapsulation: The internal implementation of a component is hidden from other components. Other components interact with it through its interface rather than directly accessing its internal code.

3 Modularity: A component should represent a relatively independent part of a larger system. A modular system can be divided into manageable parts.

4 Reusability: A component should be capable of being used in multiple systems or applications, preferably with little or no modification. For example, an authentication component could be reused in: Student management systems, Hospital systems, Banking systems, E-commerce systems

Cont.:

5 Replaceability: A component should ideally be replaceable with another component that provides equivalent services. For example, one payment provider could potentially be replaced by another without redesigning the entire application.

6 Composability: Components should be capable of being combined with other components to create larger systems.

7 Standardized interfaces: A component should expose clearly defined interfaces that specify how other components communicate with it.

8 Limited dependencies: A well-designed component should minimize unnecessary dependence on the internal details of other components.

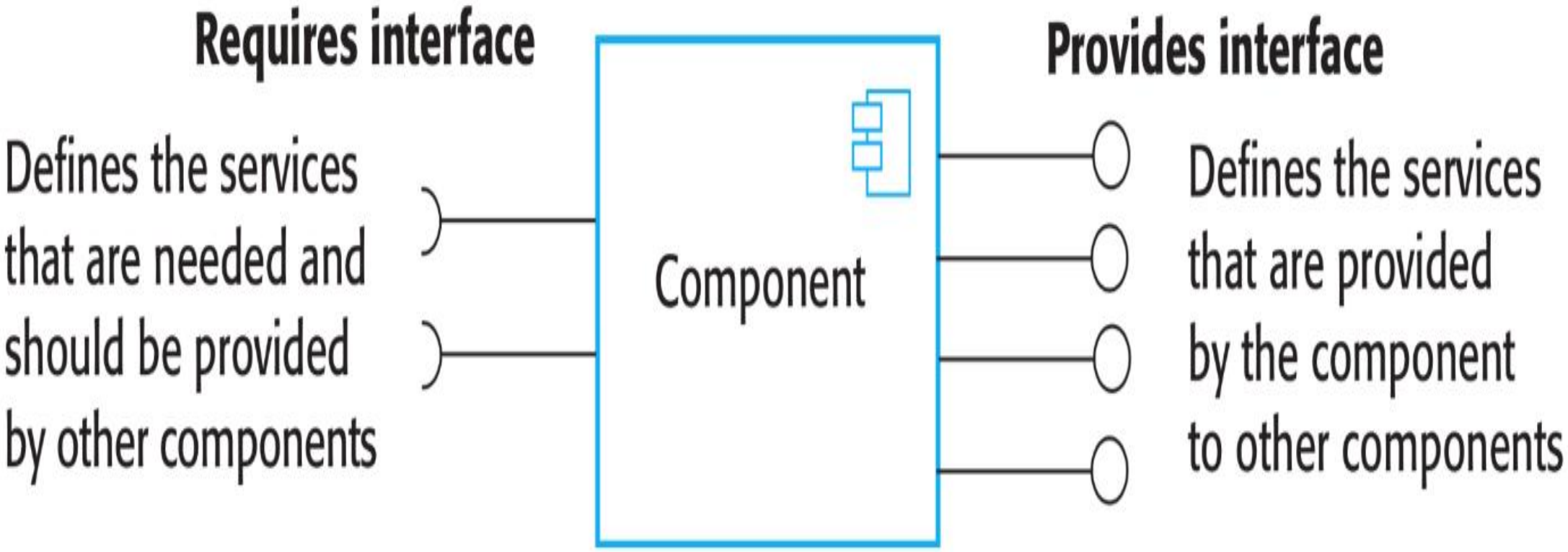
9 Deployability: A component should be capable of being deployed as an identifiable software unit within its target environment.

10 Documentation: Reusable components should be accompanied by sufficient documentation describing: Purpose, Interfaces, Inputs, Outputs, Dependencies, Configuration, Limitations and Usage requirements

Component interfaces:

- **"Provides" interface**
- Defines the services that are provided by the component to other components. This interface, essentially, is the component API. It defines the methods that can be called by a user of the component.
- **"Requires" interface**
- Defines the services that specifies what services must be made available for the component to execute as specified. This does not compromise the independence or deployability of a component because the 'requires' interface does not define how these services should be provided.

Component interfaces:

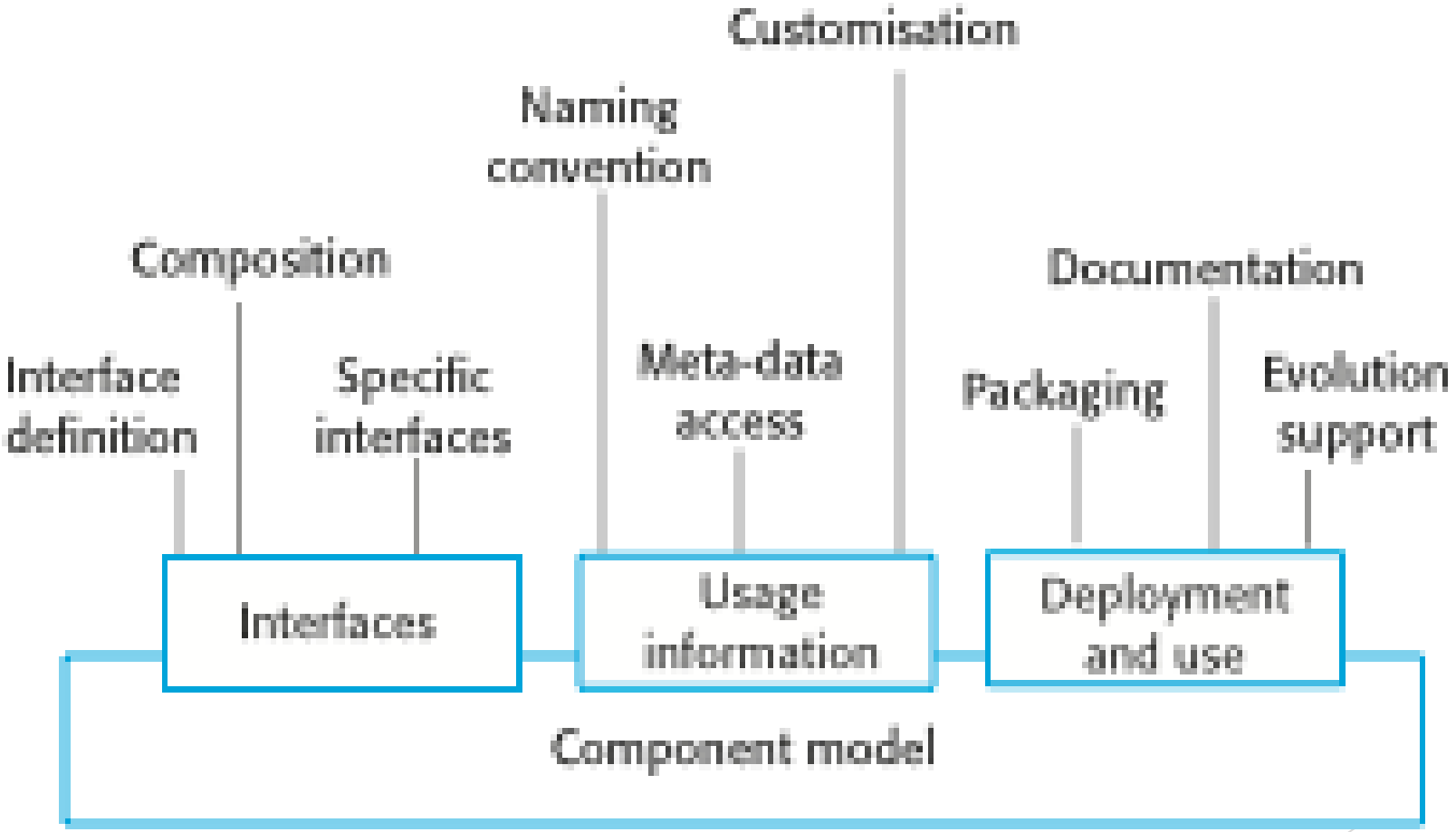


Important CBSE concepts

1: Software component

- A **software component** is a self-contained, reusable unit of software that provides specific functionality through well-defined interfaces.
- Examples include:
- Login component
- Payment component
- Shopping-cart component
- Reporting component
- Authentication component
- Database-access component

Basic elements of a component model



2. Component interface

An **interface** specifies how other software can interact with a component.

- It defines:
  - Services provided
  - Inputs required
  - Outputs produced
  - Methods/functions available
  - Communication protocols
- The implementation of the component can remain hidden.

Cont:

3 Component composition

- **Component composition** is the process of combining two or more components to create a larger software system.
- For example:
- **Login Component + Student Component + Payment Component + Reporting Component → Student Management System**

4 Component repository

- A **component repository** is a storage facility containing reusable software components and their associated documentation.
- A repository may contain:
- Source code, Executable components, Libraries, APIs, Documentation, Version information, Dependencies, Licensing information

Cont:

5 Component adaptation

- Sometimes an existing component does not exactly meet the requirements of a new system.
- **Component adaptation** involves modifying, wrapping, configuring, or otherwise adjusting a component so that it can work within the target system.

6 Component certification

- Before using a component, developers should determine whether it meets required standards.
- Certification may consider:
- Functionality, Security, Performance, Reliability, Compatibility, Maintainability, Quality

Software Components and Component-Based Systems

- **Software components**

A software component should ideally have the following characteristics:

a) **Reusability:** It can be used in more than one software system.

b) **Modularity:** It represents a relatively independent part of a system.

c) **Encapsulation:** Internal implementation details are hidden from users of the component.

d) **Well-defined interfaces:** The component communicates with other components through clearly defined interfaces.

e) **Replaceability:** A component can potentially be replaced by another component providing equivalent services.

f) **Composability:** The component can be combined with other components.

g) **Independence:** A component should have limited dependence on the internal implementation of other components.

Example: E-commerce system

An online shopping system may consist of: The complete system is produced by integrating these components.

Component	Function
Authentication	Login and user verification
Product Catalogue	Displays products
Shopping Cart	Manages selected products
Payment	Processes payments
Inventory	Tracks stock
Notification	Sends messages
Reporting	Generates reports

Evolution of Software Reuse

- Software reuse did not begin with CBSE. It has developed gradually over several decades.

1 Code reuse: Early developers reused pieces of source code by copying functions or modules from previous programs.

- **Problems**
- Difficult maintenance, Duplicate code, Strong dependencies andDifficult version management

2 Function and procedure libraries

- Developers began creating libraries containing commonly used functions.
- Examples include:
- Mathematical libraries, File-handling libraries and String-processing libraries
- This reduced repeated programming.

Cont:

3 Object-oriented reuse: Object-oriented programming introduced concepts such as:

- Classes, Objects, Inheritance, Encapsulation, Polymorphism
- These concepts improved software reuse.

4 Framework reuse

- Developers began reusing entire application frameworks rather than individual functions.
- Examples include frameworks for:
- Web applications, Mobile applications, Enterprise applications and User interfaces

5 Component-based reuse

- CBSE moved reuse toward **independently developed components** with standardized interfaces.
- Components could be developed by different teams or organizations and integrated into applications.

Cont:

- **6 Service-oriented and modern reuse:** Modern software development extends reuse through:
- Web services, APIs, Microservices, Cloud services, Package managers, Open-source libraries and Software-as-a-Service (SaaS)
- Thus, software reuse has evolved from:
- **Code → Functions → Classes → Frameworks → Components → Services/APIs → Microservices**

Objectives of CBSE

- The major objectives of Component-Based Software Engineering include:

1 Promote software reuse: Reduce the need to develop functionality that already exists.

2 Reduce development time: Using existing components can significantly reduce programming effort.

3 Reduce development cost: Less development work can result in lower software development costs.

4 Improve software quality: Well-tested and mature components can reduce defects.

5 Increase productivity: Developers can concentrate on system-specific functionality instead of repeatedly developing common features.

6 Improve maintainability: Modular systems are generally easier to modify and maintain.

7 Support flexibility: Components can potentially be replaced, upgraded, or rearranged without redesigning the entire system.

8 Encourage standardization: CBSE promotes the use of: Standard interfaces, APIs, Communication protocols, Design standards

Benefits of Component Reuse

1 Reduced development time: Existing components eliminate the need to implement functionality from scratch.

Example: A developer can use an existing authentication library instead of developing login functionality from the beginning.

2 Reduced cost: Reuse reduces: Programming effort, Testing effort, Maintenance effort, Development resources

3 Improved reliability: A component that has already been extensively tested may be more reliable than newly written code. However, developers must still verify that it is suitable for the intended environment.

4 Improved software quality: Reusable components may have undergone: Testing, Code review, Performance evaluation, Security assessment, Real-world usage

Cont.:

5 Faster development: Developers can assemble systems faster by combining existing components.

6 Easier maintenance: Changes can sometimes be made to individual components without modifying the entire system.

7 Scalability: Components can be independently improved or replaced to support changing system requirements.

8 Better productivity: Developers spend more time solving business-specific problems and less time implementing common functionality.

9 Consistency: Reusing the same components across different systems can promote consistent functionality and behaviour.

Challenges and Limitations of CBSE

- Although CBSE provides significant advantages, it also presents challenges.

1 Component identification: Finding a component that exactly matches the required functionality can be difficult. A component repository may contain hundreds or thousands of components.

2 Component compatibility: Components may have incompatible:

Interfaces, Data formats, Programming languages, Operating systems, Technologies, Communication protocols

3 Integration difficulties: Integrating components developed by different vendors or teams can be complicated.

4 Limited customization: A reusable component may not provide exactly the functionality required. Developers may have limited ability to modify it.

5 Security risks: Third-party components may contain: Security vulnerabilities, Malicious code, Outdated dependencies and Poor security practices. Therefore, components must be evaluated before deployment.

Cont:

6 Dependency management: One component may depend on several other components.

This can create a **dependency chain** that makes maintenance difficult.

7 Version problems: An updated component may introduce changes that cause existing applications to fail. This is commonly referred to as **version incompatibility** or dependency conflict.

8 Performance issues: A reusable component may perform well in one environment but poorly in another.

9 Licensing issues: Third-party and open-source components may have different licenses.

Organizations must ensure that components are used according to their licensing conditions.

10 Lack of suitable components: Sometimes no existing component satisfies the requirements. In such situations, developers may still need to develop a component from scratch.

Role of CBSE in the Software Development Life Cycle

- CBSE can influence almost every stage of the **Software Development Life Cycle (SDLC).**

1 Requirements analysis: Developers identify:

- What the system should do, Required functionality, Required interfaces and Existing components that may satisfy requirements
- **Example:** A requirement for "online payment" may lead developers to search for an existing payment component or API.

2 System analysis : Developers analyze available components and determine:

- Whether they satisfy requirements, Compatibility, Cost, Quality, Security, Licensing and Performance

3 System design: The system architecture is designed around selected components.

- Developers determine: Component responsibilities, Interfaces, Communication mechanisms, Dependencies and Data flow

Cont:

4 Component selection: Suitable reusable components are identified and evaluated.

Selection criteria may include: Functionality, Quality, Compatibility, Security, Performance, Cost, Vendor support, Documentation and Licensing

5 Component adaptation: Selected components may require: Configuration, Wrapping, Interface conversion, Parameter adjustment and Minor modification

6 Implementation: Instead of implementing the entire system from scratch, developers implement only the components that are unavailable or require customization.

7 Integration: Components are connected through their interfaces.

Cont:

8 Testing: The system is tested at several levels:

Component testing: Tests individual components.

Integration testing: Tests interactions between components.

System testing: Tests the complete system.

Acceptance testing: Determines whether the system satisfies user requirements.

9 Deployment: The completed component-based system is deployed in the target environment.

10 Maintenance: During maintenance, individual components can potentially be:

Updated, Replaced, Removed, Reconfigured and Reused in other systems

CBSE versus Traditional Software Development

Aspect	Traditional Development	Component-Based Development
Development approach	Build most functionality from scratch	Assemble existing components
Software reuse	Limited or moderate	Central principle
Development time	Usually longer	Often shorter
Cost	Can be higher	Can be lower
Quality	Depends heavily on development team	Can benefit from tested components
Flexibility	Can be highly customized	Depends on available components
Integration	Less emphasis on third-party integration	Major concern

Cont: CBSE versus Traditional Software Development

Aspect	Traditional Development	Component-Based Development
Testing	Focuses mainly on newly developed software	Includes component and integration testing
Dependencies	Usually fewer external dependencies	Can have many component dependencies
Security	Team controls most source code	Third-party components may introduce risks
Maintenance	Changes may involve large sections of code	Components can potentially be replaced independently
Architecture	Often designed around newly developed modules	Designed around component interfaces
Main focus	Software construction	Software composition and reuse

Key Principles of CBSE

- A successful CBSE approach should follow several principles:

1. **Design for reuse:** Components should be designed so that they can be used in different systems.

2. **Reuse whenever appropriate:** Developers should first investigate whether suitable existing components are available.

3. **Program to interfaces:** Systems should depend on well-defined interfaces rather than the internal implementation of components.

4. **Encapsulate implementation:** The internal details of a component should be hidden from other components.

5. **Minimize dependencies:** Components should have as few unnecessary dependencies as possible.

6. **Evaluate components before reuse:** Components should be evaluated for quality, compatibility, security and licensing.

7. **Support replaceability:** A component should ideally be replaceable by another component providing equivalent services.

Advantages and Disadvantages
Advantages

- Faster development
- Reduced development cost
- Increased productivity
- Improved reuse
- Potentially improved reliability
- Easier maintenance
- Standardization
- Reduced duplication
- Easier system evolution

Disadvantages

- Difficult component selection
- Compatibility problems
- Integration complexity
- Security risks
- Licensing concerns
- Dependency management
- Version conflicts
- Limited customization
- Performance uncertainty
- Suitable components may not exist

Recommended Reading

- Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.
- Szyperski, C., Gruntz, D., & Murer, S. (2002). *Component Software: Beyond Object-Oriented Programming* (2nd ed.). Addison-Wesley.
- Heineman, G. T., & Councill, W. T. (Eds.). (2001). *Component-Based Software Engineering: Putting the Pieces Together*. Addison-Wesley.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.
- Crnkovic, I., & Larsson, M. (Eds.). (2002). *Building Reliable Component-Based Software Systems*. Artech Hous

End

!!!!!!!